

Scanned Code Report

AUDITAGENT

Code Info

Auditor Scan

#	Scan ID 2	✦	Date February 06, 2026
📦	Organization chipi-pay	📦	Repository sessions-smart-contract
📄	Branch main	📄	Commit Hash 8251abfd...b90e129a

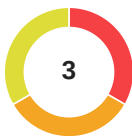
Contracts in scope

src/account.cairo src/lib.cairo tests/test_audit_fixes.cairo tests/test_session_validation.cairo
tests/test_snip9_compatibility.cairo

Code Statistics

⚙️	Findings 3	📄	Contracts Scanned 5	☰	Lines of Code 2154
----	---------------	---	------------------------	---	-----------------------

Findings Summary



Total Findings

- High Risk (1)
- Medium Risk (1)
- Low Risk (1)
- Info (0)
- Best Practices (0)

Code Summary

This protocol implements an advanced Starknet account contract built upon the OpenZeppelin `AccountComponent`. Its core feature is a robust **Session Key** management system, designed to provide granular, temporary permissions for dApps or other services without exposing the owner's primary key.

Session keys are temporary cryptographic keys that can be authorized to perform a limited set of actions on behalf of the account. The permissions for each session key are highly configurable and include:

- **Expiration Time:** A `valid_until` timestamp after which the key is no longer valid.
- **Call Limit:** A `max_calls` counter that restricts the total number of transactions the key can sign.
- **Function Whitelist:** An `allowed_entrypoints` list that specifies exactly which contract functions (selectors) the session key is permitted to call. An empty whitelist grants permission to call any function, except for a hardcoded set of administrative functions.

The contract's validation logic is bifurcated to handle both the account owner and session keys:

1. **Owner Validation:** Standard transactions are validated using a 2-element ECDSA signature (`[r, s]`) derived from the owner's private key.
2. **Session Key Validation:** Delegated transactions are validated using a 4-element signature (`[session_pubkey, r, s, valid_until]`). This path performs rigorous checks to ensure the session key is active, has not exceeded its call limit, and is only executing whitelisted functions.

For enhanced usability and gas abstraction, the account is compatible with SNIP-9 (SRC-9), enabling meta-transactions via the `execute_from_outside_v2` function. This allows third-party relayers or paymasters to submit transactions on behalf of the user. The implementation includes a critical security enhancement that enforces the session key's function whitelist *before* validating the signature, preventing unauthorized calls through this mechanism.

Furthermore, the contract includes a security blocklist that explicitly prevents session keys from accessing administrative functions such as upgrading the contract (`upgrade`) or managing other session keys (`add_or_update_session_key`, `revoke_session_key`), ensuring the owner retains ultimate control. The contract is also upgradeable and supports ERC-1271 for off-chain message signing.

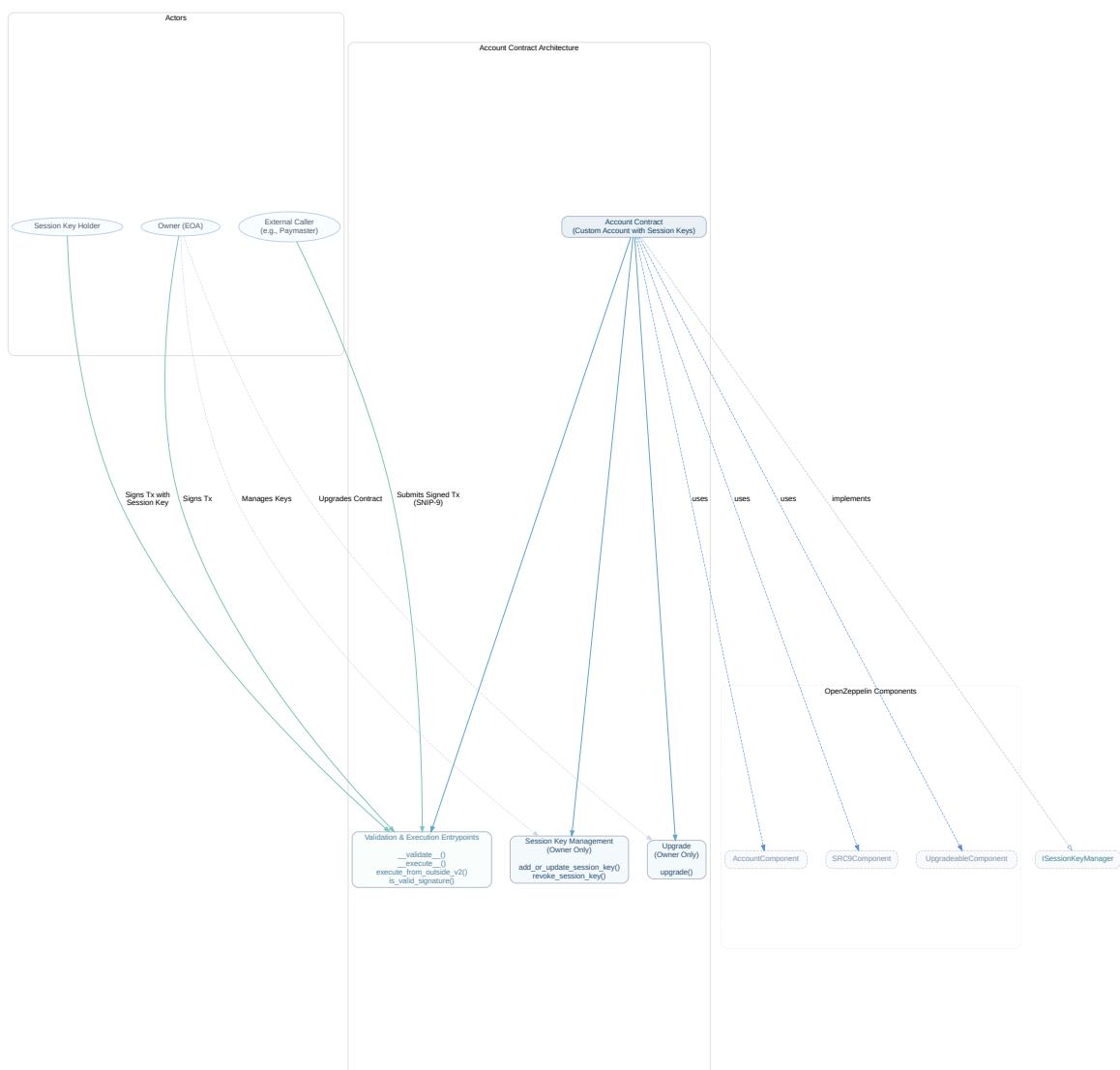
Entry Points & Actors

The primary interactions with the account's state are managed through the standard Starknet account entry points and a specialized function for meta-transactions.

- **Actors:**
 - **Starknet Sequencer:** The network entity that processes transactions and invokes the core validation and execution functions.
 - **Account Owner:** The user who holds the main private key for the account. They sign transactions to be executed directly or to authorize outside execution.
 - **Session Key Holder:** A delegated entity (e.g., a dApp) that holds a temporary key with limited permissions. They sign transactions using the session key.
 - **Relayer:** A third-party service that submits pre-signed messages to the network, typically for meta-transactions or gas abstraction.
- **Entry Points:**

- `__validate__`: Validates a transaction by checking the signature. It supports both owner signatures and session key signatures, applying the corresponding permission checks for the latter.
- `__execute__`: Executes an array of calls after a transaction has been successfully validated.
- `__validate_deploy__`: Validates the parameters during the account's deployment.
- `__validate_declare__`: Validates a declare transaction initiated by the account.
- `execute_from_outside_v2`: Allows a relay to execute a transaction on behalf of the owner or a session key holder by providing a valid signature. This function consumes a nonce to prevent replay attacks and enforces all session key restrictions.

Code Diagram



✦ 1 of 3 Findings

src/account.cairo

Session keys can bypass whitelist/admin blocklist by calling the account's own execute (nested execution), enabling upgrade and session management**• High Risk**

The session-key permission system only validates the *top-level* `calls` array passed to `__validate__` / `execute_from_outside_v2` by checking each `call.selector` against an admin blocklist and (optionally) a selector whitelist. However, session keys are not prevented from calling the account's own `__execute__` entrypoint as a subcall.

Because `__execute__` explicitly allows calls when `get_caller_address() == get_contract_address()`, a session can make a top-level call to `__execute__` (which passes the selector checks), and then provide an arbitrary nested `Array<Call>` in the calldata that `__execute__` will execute with the account as the caller. This enables executing selectors that are otherwise blocked for sessions (including `upgrade`, `add_or_update_session_key`, and `revoke_session_key`) and also enables executing selectors not present in a non-empty whitelist.

Key code paths:

1) Session enforcement checks only the outer selectors and returns early when the whitelist is empty:

```
// _is_session_allowed_for_calls
// First pass: check for blocked admin selectors
if sel == UPGRADE_SELECTOR
    || sel == ADD_SESSION_SELECTOR
    || sel == REVOKE_SESSION_SELECTOR {
    return false;
}

// If no whitelist (allowed_entrypoints_len == 0), allow all non-admin selectors
if session.allowed_entrypoints_len == 0 { return true; }
```

2) The account's `__execute__` allows the account itself to call it:

```
// __execute__
let caller = get_caller_address();
assert(
    caller.is_zero() || caller == get_contract_address(),
    "Account: unauthorized caller"
);

self._execute_calls(calls)
```

3) `__execute_calls` uses `call_contract_syscall`, so a validated session transaction can include a call to the account itself:

```
// _execute_calls
starknet::syscalls::call_contract_syscall(call.to, call.selector, call.calldata)
```

Exploit sketch (session with empty whitelist, or with `__execute__` whitelisted):

- Submit a session-signed transaction whose *only* top-level call is:
 - `to = get_contract_address()`
 - `selector = selector!("__execute__")`
 - `calldata = <ABI-encoded Array<Call> containing admin calls>`
- `_is_session_allowed_for_calls` inspects only the top-level selector `__execute__` (not in the admin blacklist) and, for empty whitelist, returns `true`.
- During execution, `_execute_calls` performs a syscall to the account itself; inside that nested call, `get_caller_address()` equals the account, satisfying `__execute__`'s caller check.
- The nested `__execute__` executes the attacker-controlled inner calls, including admin functions that are intended to be permanently blocked for sessions.

Impact:

- Session keys can effectively obtain owner-equivalent privileges (upgrade the account, add/update/revoke sessions, and execute any non-whitelisted selector by nesting it), breaking the intended session isolation model.
- This violates the intended guarantee that session keys cannot access administrative selectors or escape their whitelist constraints.

Severity Note:

- The targeted account has at least one active session key (valid, within `max_calls`).
- The session either has an empty whitelist or explicitly whitelists **execute**.
- Standard AA flow passes the provided top-level calls unmodified to **validate/execute**.

✦ 2 of 3 Findings

src/account.cairo

Session-signature validation in validate does not authenticate the full Starknet transaction hash (fee/paymaster/resource-bound fields are not covered)

• Medium Risk

The session-signature path in `__validate__` verifies ECDSA over a custom Poseidon hash computed by `_session_message_hash(calls, valid_until)`, rather than the protocol transaction hash (the one that typically commits to the full transaction envelope).

Hash construction:

```
fn _session_message_hash(
    self: @ContractState,
    calls: Span<Call>,
    valid_until: u64
) -> felt252 {
    let tx_info = get_tx_info().unbox();
    let mut hash_data = array![];

    hash_data.append(get_contract_address().into());
    hash_data.append(tx_info.chain_id.into());
    hash_data.append(tx_info.nonce.into());
    hash_data.append(valid_until.into());

    // then appends each call's (to, selector, calldata_len, calldata...)
    ...

    poseidon_hash_span(hash_data.span())
}
```

Notably, the hash does not include any commitment to other transaction-level parameters that can affect execution viability and the account's costs, such as (depending on Starknet tx version and features): max fee / resource bounds / tip, and any paymaster-related envelope fields.

As a result, any party that has obtained a valid session signature for a given

`(calls, nonce, chain_id, valid_until)` can submit a Starknet transaction that keeps those signed fields identical but alters non-signed envelope fields. While this does not allow changing the called contracts/selectors/calldata (those are signed), it can allow relayer-side tampering with fee/payment-related parameters in a way the session signer did not explicitly authorize.

This is a security-relevant integrity gap for the session-signature `__validate__` path because the account contract is accepting `starknet::VALIDATED` (1) based on a signature that does not necessarily bind to the full transaction context under which the sequencer will execute and charge for the transaction.

Severity Note:

- Starknet tx version in use allows modifying fee-related envelope fields (e.g., max_fee/resource bounds/tip/paymaster fields) independently of calls.
- Increasing certain envelope fields (e.g., tip) can increase what the account actually pays if the attacker's resubmission is included first.
- A relayer or observer can see and reuse the session signature before inclusion.

✦ 3 of 3 Findings

src/account.cairo

Session call consumption before signature validation in execute_from_outside_v2 causes off-by-one error

• Low Risk

In the `execute_from_outside_v2` function, when processing a session signature (4-element signature), the function calls `_consume_session_call(session_pubkey)` to increment `calls_used` BEFORE calling `is_valid_signature` for signature validation. However, `is_valid_signature` also checks `session.calls_used >= session.max_calls` as part of its validation logic.

This creates an off-by-one error where the incremented `calls_used` value causes signature validation to fail:

```
// In execute_from_outside_v2:
if signature.len() == 4 {
    let session_pubkey = *signature.at(0);
    assert(
        self._is_session_allowed_for_calls(session_pubkey, outside_execution.calls),
        'Session: unauthorized selector'
    );
    self._consume_session_call(session_pubkey); // calls_used incremented HERE
}
// ...
let is_valid_oz = SRC6Impl::is_valid_signature(@self, oz_hash, sig_copy1); // checks
calls_used HERE
```

```
// In is_valid_signature for sessions:
if session.calls_used >= session.max_calls { return 0; } // Fails because calls_used was
already incremented
```

For a session with `max_calls = 1` and `calls_used = 0`:

1. `_is_session_allowed_for_calls` passes ($0 \geq 1$ is false)
2. `_consume_session_call` sets `calls_used = 1`
3. `is_valid_signature` checks $1 \geq 1$ which is true, returns 0 (invalid)
4. Transaction fails

This means sessions with `max_calls = 1` can NEVER use `execute_from_outside_v2`, and all other sessions effectively lose one call when using this path. This breaks paymaster/relayer functionality for single-use session keys.

In contrast, `_validate` correctly calls `_consume_session_call` AFTER signature verification succeeds.

Disclaimer

Kindly note, no guarantee is being given as to the accuracy and/or completeness of any of the outputs the AuditAgent may generate, including without limitation this Report. The results set out in this Report may not be complete nor inclusive of all vulnerabilities. The AuditAgent is provided on an 'as is' basis, without warranties or conditions of any kind, either express or implied, including without limitation as to the outputs of the code scan and the security of any smart contract verified using the AuditAgent.

Blockchain technology remains under development and is subject to unknown risks and flaws. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. Neither you nor any third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.